

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 28, 2026 (2 hour)

Objective:

Speed up queries by adding threading then testing it for faster times

Attempts made:

Split the queries into equal parts for each thread.

```
int chunk = tasks.size() / num_threads;

for (int t = 0; t < num_threads; t++) {

    int start = t * chunk;

    int end = (t == num_threads - 1) ? tasks.size() : start + chunk;

    threads.emplace_back([&, start, end]() {

        for (int i = start; i < end; i++)

            tasks[i].result = tasks[i].cb(g, tasks[i].src, tasks[i].K);

    });

}
```

The problem is queries aren't all the same cost. A source vertex with tons of connections at $K=3$ takes way longer than one with barely any neighbors. Some threads would finish early and just sit there while one was still grinding through heavy queries.

Solutions/Workarounds:

Used an atomic counter instead so threads just grab whatever's next until there's nothing left:

```
atomic<int> next_task{0};

auto worker = [&]() {

    while (true) {

        int idx = next_task.fetch_add(1, memory_order_relaxed);
```

```
    if (idx >= (int)tasks.size())  
  
        Break;  
  
    tasks[idx].result = tasks[idx].cb(g, tasks[idx].src, tasks[idx].K);  
  
}  
  
};
```

Each task only writes to its own result so no mutex needed. Checked that sequential and parallel gave the same answers before looking at timing.

Commits:

Add parallel task runner with atomic work queue

Add timing and correctness reporting

Experiments/Analysis:

Ran queries10000.txt on soc-Slashdot0902.txt with 4 threads:

Sequential time : 4799.36 ms

Parallel time : 3680.99 ms

Speedup : 1.30x

Correctness : ALL PASS

1.30x with 4 threads is pretty low. My guess is BFS on this graph is mostly just reading memory, edges[] and offsets[] are big and multiple threads reading them at the same time compete for bandwidth. Some queries at K=3 on a dense social network probably visit a huge chunk of the graph and there's no way to speed up a single one of those regardless of thread count.

Learning outcome:

Atomic counter for work distribution is simple and works better than static chunking when tasks have uneven cost. Also didn't realize memory bandwidth could be such a bottleneck — more threads doesn't help if they're all just waiting on RAM reads.